An Algebraic Approach to Cheminformatics with the UMOL Library

Dmitrij Rappoport[1,2]

[1]Dayhoff Labs, Inc., Cambridge MA 02140, USA
[2]Department of Chemistry, University of California, Irvine CA 92697, USA

dmitrij@rappoport.org

October 26, 2026

Abstract

The attributed undirected graph of atoms and localized bonds underlies cheminformatics, reaction network studies, and much of the machine-learning work in chemistry. Several parts of chemistry are standing exceptions to it. Stereo descriptors depend on a neighbor ordering the graph leaves immaterial, and they change under modifications far from the atom carrying them; the aromatic markers on ring atoms are coupled to one another; and compounds whose bonding does not fit the localized model, among them triplet dioxygen, diborane, and ferrocene, have no faithful representation. Such exceptions are usually read as facts about chemistry. We suspect they are facts about the model. This paper describes UMOL, a Rust library with Python bindings, built on two extensions to that model. Typed attributed hyperedges, called overlays, carry aromatic systems, stereo atoms and bonds, and dative, multicenter, and noncovalent bonds, leaving the topology an ordinary attributed graph. The attributes of atoms, bonds, and overlays are placed in a lattice, so that a concrete value, a set of admissible values, and an undetermined value are points of one order. Matching, resolution, and validation become comparisons in that order, and molecules, structural patterns, and reactions share one notation. A reaction is a left-hand side together with an ordered list of deltas, and reactions compose and apply by the double-pushout construction, with the dangling condition extended to overlays. The rules of manipulation are explicit, so algebraic laws hold exactly and uniformly: manipulation preserves molecule integrity by construction, duplicates are recognized, and reaction rules compose. We describe the design, the notation, and the trade-offs it accepts.

1 Introduction

In this paper, "algebra" is taken to mean both the specific subfields of modern algebra relevant to the theory of chemical structure, namely, graph theory, relational algebra, and theory of posets and lattices, and the broader notion of algebra as the manipulation of objects according to a set of prescribed laws. For the purposes of this paper, algebra thus defines what chemical objects are, which operations they allow,

and how these operations compose. The standard representation of molecular structures is the attributed undirected graph, with atoms as nodes and localized bonds as edges. It underlies cheminformatics, automated synthesis prediction, reaction network studies, and many other areas, including a large portion of AI applications in chemistry. This particular conception of molecular structure is inherent in the textual representations of molecules (SMILES [1, 2], MOL [3]), substructures (SMARTS), and reactions (SMIRKS) [4], in the data models of cheminformatics software (RDKit [5], ChemAxon [6], OpenBabel [7], CDK [8], Indigo [9], MØD [10]), main chemical databases (CAS SciFinder [11], PubChem [12], Reaxys [13], ChemSpider [14], ChEMBL [15], ZINC [16]), and model training datasets (GDB-13 [17], GDB-17 [18], USPTO [19], Transition1x [20], OMol25 [21]).[1] It is the proverbial water we swim in.

Although graphs have served as molecular representations for over 150 years [22, 23] and are considered the natural language of chemistry, this understanding always came with an asterisk. Take L-histidine as an example, a key proteinogenic amino acid. The α-carbon stereocenter is denoted by the counterclockwise arrangement of its stereo ligands if we assign their ordering according to the CIP sequence rule [24]. However, swap two ligands, or reduce the carboxyl group to a methyl group, and the notation changes to clockwise. In the second case, the atom bearing the attribute is unchanged but the attribute has to adjust to a non-local modification. This is intuitive for a trained chemist but it is a rule violation for attributed graphs, in which the neighbor ordering is immaterial and an attribute cannot depend on a distant part of the structure. The aromatic imidazole ring requires special treatment, too; the aromatic markers on the ring atoms are coupled to each other, which makes graph rewriting rules in aromatic systems dependent on ring size, in deviation from ordinary graph rewriting. In compounds with more than one ring, such as naphthalene or biphenyl, atom markers cannot convey how many disjoint aromatic systems are present and which ones are affected by graph rewriting. Finally, many well-characterized chemical compounds are simply invisible to the graph model. Such cheminformatics "dark matter" includes perfectly stable compounds whose bonding types do not fit the localized bond model, for example, triplet O_2, diborane B_2H_6, coordination compounds such as heme B and ferrocene $Fe(C_5H_5)_2$. Some cheminformatics codes provide ad hoc representations for these bonding situations but no well-defined manipulation rules.

These issues are well known to chemists: aromaticity, stereochemistry, and transition-metal compounds are the parts of chemistry that practitioners agree are "hard". However, these standing exceptions are usually taken as a fact about the chemistry and not the model. We suspect that the causation runs the other way. There are good reasons to want algebraic laws to apply exactly and uniformly: the ability to generate guaranteed valid molecular structures [23], identify structural duplicates [25], or write reaction rules that compose [10] is what makes unsupervised computational exploration possible. However, as the above examples show, law adherence is not achievable within the molecular graph model and, because the mismatch is fundamental to the model, cannot be brought about by improved tooling, validation, or model training.

The influence of the molecular graph model can be observed throughout the cheminformatics and computational chemistry ecosystem. The major cheminformatics libraries do not just store a molecular

[1] Product and company names mentioned in this paper may be trademarks or registered trademarks of their respective owners and are used for identification and comparison only, without implying endorsement.

graph; they enforce a model of what a molecule may be at the point of input. Normalization (referred to as *sanitization* in RDKit [5], the library synonymous with open-source cheminformatics) rewrites structures with unusual valence, runs aromaticity perception as a matter of course, and applies valence rules that decide which inputs count as valid. This notion of valid molecules, which is strongly biased toward drug-like molecules, runs through the datasets on which machine-learned models are trained. The chemical space enumerated by the GDB databases [17], built to a fixed set of valence rules, was distilled into the QM7 [26] and QM9 [27] datasets, which in turn underlie reaction and transition-state collections such as those of Grambow et al. [28] and Transition1x [20]. A valence model chosen for database enumeration in 2009 thus persists, largely unexamined, in the data used to train today's machine-learned interatomic potentials, and its biases propagate quietly upward through featurizers, generative models, and benchmarks. The most recent datasets such as Open Molecules 2025 [21] and the universal interatomic potentials trained on it [29] incorporate the earlier transition-state data in their reaction training subset.

The key proposition of the UMOL library[2] and of this paper is that two extensions are sufficient to make the molecular graph model fulfill algebraic laws concerning its topology and atom and bond attributes. The first augments the molecular graph with a set of typed, attributed hyperedges to represent the features that do not fit into the graph of atoms and localized bonds, including aromatic systems, stereo atoms and stereo bonds, coordination and multicenter bonds, among others. Hyperedges [30] are a generalization of ordinary graph edges and may connect any number of nodes instead of exactly two. We call these hyperedges *overlays*, and, following the relational reading developed below, treat each kind as a *relation set*. The second addition places the attribute values of atoms, of bonds, and of overlays into a lattice, an ordering that lets a single vocabulary serve for concrete molecules, structural patterns, and reactions alike. Two terms will recur throughout. We use *molecular graph* and *molecular topology* interchangeably for the attributed undirected graph of atoms and localized bonds, reserving *bond*, unqualified, for its localized edges; a *molecular structure* is then a molecular topology together with its overlays.

The split of the molecular topology and overlays simplifies them both. Because aromaticity and stereochemistry are lifted out of the graph and into overlays, what remains is an ordinary attributed graph with well-defined semantics, including graph rewriting, which the special-case treatment of aromatic and stereo labels would otherwise obstruct. The molecular topology also stays maximally compatible with the standard structural model of cheminformatics and allows straightforward mapping to the existing body of data and tooling. On the other hand, the overlays correspond to a relation set, or a table in the language of relational databases. The foreign keys of that table refer to the atoms and bonds that participate in the overlay, which the molecular graph model describes, for example, as aromatic atoms or chiral atoms. In this way, we recognize the per-atom and per-bond annotations (aromaticity or stereo parity) as *materialized projections* of the overlays onto their participants. Keeping the localized bonds in the graph instead of folding them into the relational model is an engineering choice: graph traversal is a primitive operation on a graph but a sequence of table joins on a set of relations, and the split between the two mirrors the everyday distinction between graph and relational databases.

[2] The recommended pronunciation for the library name is micromole (μmol), commonly rendered as umol in plain text.

The *attribute lattice* addresses a different concern about the language of chemistry, which is divided between molecules (for example SMILES), the molecular patterns for substructure matching (SMARTS), and rewriting rules for reactions (SMIRKS). Unifying these notations into one is known as *homoiconicity*, to borrow a term from functional programming languages, especially those in the LISP lineage. We note that SMILES and SMARTS are already homoiconic since every valid SMILES expression is also a valid SMARTS expression [4]. The attribute lattice goes one step further: each node, edge, or hyperedge attribute can be a literal value, a set of admissible values, or an undetermined value (wildcard). Examples of attributes are chemical element, charge, number of unpaired electrons, or number of lone pairs for atoms, bond order, charge, and spin for bonds, atomic π electron contributions for aromatic systems, ligand orientation for stereo atoms and bonds. The attribute values are then arranged in a set-inclusion lattice, an algebraic structure with well-defined rules [31] and recognizable semantics. Matching a pattern against a molecule becomes a comparison in the lattice: the molecule matches when it *refines* the pattern. Resolving a partially defined molecular structure from an external source requires filling in hydrogen counts on the organic subset, unpaired electrons, and lone pairs. With the lattice structure present, *resolution* becomes refinement in the same order, which carries an underspecified structure toward a concrete molecule, also referred to as a *ground term*. Constraints such as atom degree and valence, familiar from SMARTS, are projections of the graph and overlay relations onto atoms, and are tested by the same comparison. Reactions, finally, are represented by a left-hand side together with an ordered list of changes, or *deltas*. Since the attributes live in the lattice, a concrete reaction and a reaction rule share one representation, and their composition and application follow the double-pushout construction of graph rewriting [32], with the dangling condition that governs deletion extended to cover overlays.

Before turning to the basic principles and the functioning of UMOL, we acknowledge the theoretical and implementation work on which it draws. Adding higher-arity relations to the molecular graph (XBE) was proposed by Ugi and coworkers [33] as an extension of their bond-electron (BE) matrix notation [34]. Dietz suggested to use a flat notation for delocalized and multicenter bonds [35] instead of hyperedges. Both XBE and the Dietz notation were primarily designed as tools for structure representation. More recently, molecular structure has been recast as an algebraic data type [36]. The double-pushout construction and its dangling condition come from the theory of algebraic graph transformation [32], and their application to chemistry was pioneered by the MØD system [10], which remains the inspiration for this work; the treatment of graph attributes as lattice-valued constraints follows the symbolic and attributed graph-transformation literature [37]. The relational and homoiconic treatment of structure draws on ideas long established in functional programming and in relational and logic-based data models. The chemf toolkit [38] showed the possibility of implementing chemistry software in the functional programming style.

The contributions of UMOL are the separation of a localized graph from relational overlays, the lattice-valued attributes carried on both, and the single homoiconic representation of molecules, patterns, and reactions that the combination makes possible. Taken together, they bring “dark matter” compounds, including open-shell and electron-deficient compounds, delocalized and multicenter bonding, and coordination complexes, within reach of the same well-defined operations as ordinary molecules, and let molecules, patterns, and reactions be treated as one kind of object with algebraic laws that hold exactly.

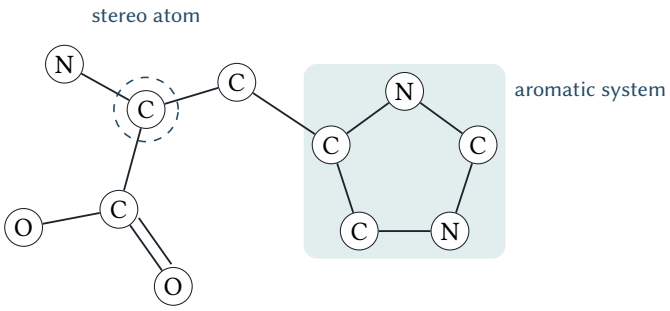


Figure 1. The molecular topology (atoms and localized bonds) and overlays (aromatic system of the imidazole ring and the stereo atom) of the L-histidine molecule. The ring bonds are the ordinary single bonds of the σ scaffold; the delocalization is represented by the aromatic system overlay. The stereo atom is of tetrahedral stereo type and includes the α-C site and the β-C, carboxyl C, N atoms, and an implicit H atom as stereo ligands.

This does not make the "hard" parts of chemical structure suddenly easy; it does, however, isolate the complexity and give explicit rules of manipulation instead of operational definitions. The remainder of the paper proceeds as follows. Section 2 discusses the basic principles underpinning UMOL. Section 3 describes the notation, an EDN-based external envelope with a compact string sublanguage for attributes, and Section 4 is a short primer in the Python and Rust interfaces. Sections 5 to 10 then develop the molecular topology and overlays, structural identity and the constraints derived from them, validity and the chemistry model, the attribute lattice, mutation, and reactions in turn; appendices give the Rust interface, a glossary, and a reference summary of the notation.

2 Basic Principles

1. Molecular Topology. The molecular graph is an attributed undirected simple graph (no self-edges or parallel edges) with atoms as its nodes and localized bonds as its edges [39–41]. Localized bonds are the single, double, and triple bonds that are not part of a delocalized system; this includes the σ scaffold of aromatic rings as well as the isolated double and triple bonds of ethene and ethyne. For brevity, bond without further qualification always means a localized bond. Everything that does not fit this picture, including delocalization, stereochemistry, coordination, is carried by the overlays introduced next, so the topology itself remains an ordinary attributed graph.

2. Relational Overlays. Everything lifted out of the topology is represented by typed, attributed hyperedges [30], which we call overlays. A hyperedge joins any number of atoms rather than exactly two, and each family of overlays behaves as a relation set, denoted as a table in the theory of relational databases [42, 43], whose rows record the participating atoms together with overlay attributes (Figure 1). The participants may be unordered, as in an aromatic system, or ordered, as in a stereo atom or stereo bond, and the same construction covers coordination and multicenter bonds. The attributes carry, for example, the per-atom π-electron contributions of an aromatic system or the orientation label of a stereo element (the orbit of the ligand permutation group under proper rotations).

3. Identity. Two molecular structures are the same when their topologies are isomorphic under a relabeling of atoms that also matches all attributes and overlays; the atom numbering itself carries no meaning, in keeping with the convention that graphs are identified up to isomorphism [44]. A canonical labeling turns this equivalence into a decidable equality test, which is what makes structures hashable and lets duplicates be detected [25]. Because aromaticity and stereochemistry have been moved into overlays, the topology is a proper attributed graph, and its canonical form can be computed with a standard graph-canonicalization algorithm [45], removing the aromaticity or stereo perception step as a prerequisite. That step is a hidden source of disagreement between toolkits: because the canonical form depends on which aromaticity model was applied, the "same" molecule can canonicalize differently under different models. In the split representation, the topology canonicalizes independently of any such model, and the overlays are canonicalized on top of it.

4. Homoiconicity. Molecules and the patterns that match them are written in the same notation, differing only in how far their attributes are pinned down. We borrow this property and terminology from the Lisp family of languages and from Clojure in particular [46]. An attribute holding a concrete value (a literal, or ground term) belongs to a molecule; the same attribute holding a set of admissible values, or an unspecified wildcard, belongs to a pattern. Thus C names a carbon atom, $\{\mathrm{C}, \mathrm{N}, \mathrm{O}\}$ a pattern position matching any of the three, and $*$ any atom at all. Reactions extend the same idea: a reaction is a left-hand side together with a list of changes (deltas), so a concrete reaction and a reaction rule are once more one kind of object. A single vocabulary therefore serves molecules, substructure queries, and reactions alike.

5. Attribute Lattice. The attribute values form a partially ordered set (poset) under set inclusion [31]. The chemical element, for instance, is ordered as $\mathrm{C} \prec \{\mathrm{C}, \mathrm{N}, \mathrm{O}\} \prec *$, where $*$ is the element wildcard and $\prec$ denotes the partial ordering relation (Figure 2). A poset differs from a total ordering such as the natural numbers in that not all pairs of objects are comparable: $\mathrm{C} \not\prec \mathrm{N}$ and $\mathrm{N} \not\prec \mathrm{C}$. For two elements a and b of a poset $\mathscr{P}$, the meet $a \wedge b$ is their greatest lower bound ($a \wedge b \prec a$ and $a \wedge b \prec b$) and the join $a \vee b$ their least upper bound. For $a = \{\mathrm{C}, \mathrm{N}\}$ and $b = \{\mathrm{N}, \mathrm{O}\}$, the meet is $a \wedge b = \mathrm{N}$ and the join is $a \vee b = \{\mathrm{C}, \mathrm{N}, \mathrm{O}\}$; when both exist for every pair, the poset is a lattice. A pattern a matches a value b when $a \wedge b = b$, that is, when b refines a. This order is defined on attribute values, and componentwise on individual atoms, bonds, and overlays. Topology matching uses subgraph isomorphism; attribute matching uses the lattice order.

6. Constraints. Molecular patterns like SMARTS constrain atomic properties such as degree, valence, and the like. These properties are projections of the edge and hyperedge relations onto the atoms [43]. The degree of an atom is the number of incident edges; its localized valence is the sum of the incident bond orders. The same construction extends to overlays: the aromatic valence of an atom is its π-electron contribution to an incident aromatic system. In the imidazole ring of Figure 1, the aromatic valence of the pyridine-type nitrogen is one and that of the pyrrole-type nitrogen is two, and summing the contributions around the ring gives the six π electrons of an aromatic sextet. The same projection may

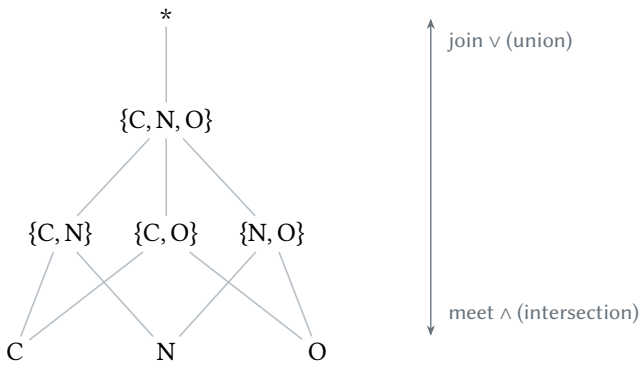


Figure 2. Part of the attribute lattice for the chemical element, ordered by set inclusion: single elements at the bottom, their unions above, and the wildcard $*$ as the top element. The meet of two values is their intersection (downward) and the join their union (upward). The empty-set bottom is omitted.

be asserted as a constraint in a pattern or computed from a concrete structure, and in either case it is compared in the sense of the attribute lattice.

7. Validity. The question whether a molecule representation is valid has two parts. One verifies physical invariants such as electron counts per atom, bond, or overlay, which must be satisfied by all molecules, regardless of structural model. The other is model-dependent validation, which applies a defined *chemistry model* including the choice of valence rules, aromaticity definitions, and admissible stereo kinds. The common organic chemistry conventions are encoded as SMILES normal valence rules [2] and as Daylight and other aromaticity models [4, 5]. They constitute one particular set of definitions: some set of rules might reject diborane B_2H_6 or ferrocene $Fe(C_5H_5)_2$ molecules, while another chemistry model would validate them.

8. Delta Algebra. A delta encodes a change to a structure: adding or removing atoms, bonds, or overlays, or modifying their attributes. These are the primitive operations of graph and hypergraph rewriting. Each delta denotes a function on structures, and an ordered list of deltas denotes the composition of those functions, which are executed in order. Every delta has an inverse, which is again a delta, and inverting twice returns the original. A principal use of deltas is in defining reactions.

9. Double-Pushout Rewriting. A reaction is a left-hand side (LHS) molecule together with an ordered list of deltas. Both the LHS molecule and the deltas share attribute definitions and the attribute value lattice; as a result, concrete reactions and reaction rules are homoiconic. Reactions compose into one or more combined reactions, and a reaction applies to a molecule to yield one or more products. Throughout, an extended dangling condition holds: a rule deleting an atom is admissible only if it also deletes every bond and overlay in which that atom participates. Removing the acidic hydrogen of a carboxyl group, for instance, must also account for the bond that hydrogen terminated. This is the double-pushout construction of algebraic graph rewriting [32], whose application to chemistry was established by the

```
{:atoms [[:O "O"] [:C "C"] [:CA "C#h"] [:CB "C#h2"] [:CG "C"] [:CD2 "C#h"]
         [:NE2 "N#h"] [:CE1 "C#h"] [:ND1 "N"] [:N "N#c+#h3"] [:OXT "O#c-"]]
 :bonds [[:O :C :double] [:C :CA :single] [:CA :CB :single] [:CB :CG :single]
         [:CG :ND1 :single] [:CG :CD2 :single] [:CD2 :NE2 :single]
         [:NE2 :CE1 :single] [:CE1 :ND1 :single] [:CA :N :single] [:C :OXT :single]]
 :aromatic-systems [{:atoms [:CG :CD2 :NE2 :CE1 :ND1] :attrs "[1,1,2,1,1]"}]
 :stereo-atoms [{:site :CA :ligands [:C :CB :N [:h :CA]] :attrs :ccw}]}
```

Figure 3. L-histidine zwitterionic form of Figure 1 in the UMOL notation. Keywords `:O`, `:C`, ...denote atom ids. Localized bond orders are written in keyword notation `:single`, `:double`. The imidazole aromatic system is defined along with per-atom π-electron contributions. The α-C stereo atom includes the orientation with respect to the fixed ligand ordering.

MØD system [10] and extended here so that a deletion may not orphan an overlay. The proton transfer $NH_3 + HCl \longrightarrow NH_4^+ + Cl^-$ is written as such a rule in Figure 4.

3 Notation

UMOL uses a domain-specific language (DSL) as an external representation of its data model with two layers: an outer envelope in the extensible data notation (EDN) [46, 47], which encodes its shape, and a compact string sublanguage that fills in the per-entity attributes. EDN is the data notation developed for the Clojure language [46] and used to express database queries [48], among other applications. In contrast to other structured data formats, it uses keywords, for example, `:atoms`, `:bonds`, `:stereo-atoms`, to identify structural elements rather than using strings as JSON does. Its collection syntax makes nested structures explicit. However, EDN is far less widely implemented than those formats, and UMOL provides its own EDN parsing and rendering library to support the notation. For the full EDN syntax documentation, see [47].

A molecule is a single EDN map keyed by relation keywords. Maps are equivalent to objects in JSON or dicts in Python and are written between curly braces { }. Vectors are like arrays in JSON or Python lists and use brackets [ ]. Whitespace serves as a separator throughout. Two keys are required, `:atoms` and `:bonds`. The overlays contribute their own: `:aromatic-systems`, `:stereo-atoms`, `:dative-bonds`, etc. The `:atoms` vector contains atom definitions in the string sublanguage (`"C#h3"`), with optional identifiers to simplify symbolic references (`[:C "C#h3"]`) in bonds and overlays. `:bonds` is a vector of triples with the atom references and the bond type string. Overlays specify their participating atoms by role and carry an `:attrs` payload. Figure 3 shows L-histidine zwitterion in the DSL notation, including the aromatic system and stereo atom notations.

Inside the strings is the second layer. An atom string begins with the element, followed by any number of predicates, marked by single-letter hash tags: `C#h3` is a carbon with three implicit hydrogens and `N#c+` a nitrogen with a positive formal charge. Inherent atom fields (`#c` charge, `#h` implicit hydrogens, `#n` lone pairs, `#u` unpaired electrons and `#s` spin multiplicity) and constraints (`#v` valence, `#a` aromatic valence, `#D` degree, `#H` total hydrogens) are represented by the same syntax. The value one following numerical

Table 1. Selected attribute strings for atoms, bonds, and overlays.

Entity	String	Meaning
Atom	`"C"`	carbon, no further attributes fixed
	`"C#h3"`	three implicit hydrogens
	`"N#c+"`	a positive formal charge
	`"C#h*"`	hydrogen count unconstrained (pattern)
	`"{C,N}"`	element carbon or nitrogen (pattern)
	`"!H"`	element not hydrogen (pattern)
	`"*#h2"`	element wildcard, two implicit hydrogens (pattern)
Bond	`"1"` / `:single`	single bond
	`"1#a"` / `:aromatic`	single bond coincident with aromatic system
	`"*"`	undetermined order (pattern)
Aromatic system	`"[1,1,2,1,1]"`	π contributions by atom
	`"*"`	π contributions undetermined (pattern)
Stereo atom	`"Th0"` / `:ccw`	tetrahedral, counterclockwise
	`"Th*"`	tetrahedral, ordering undetermined (pattern)

fields or constraints can be omitted, that is, `0#h` and `0#h1` notations are equivalent.

The bond string consists of its order and constraints (`1`, `2`, `3` for localized single, double, and triple bonds, respectively, `1#a` for a localized bond with an aromatic constraint). Because the localized bonds in an aromatic ring only include the σ-scaffold, their localized bond order is always 1. Common bond types can alternatively be written as keyword shorthands (`:single`, `:double`, `:triple`, `:aromatic`).

An overlay's `:attrs` string carries the overlay attributes such as the atomic π-electron contributions in an aromatic system in the order of the participating atoms. The stereo atoms and bonds carry their kind and configuration. Some examples are shown in Table 1, and Appendix C is a reference summary of the grammar; the normative specification is distributed with the library, as is a formalization of the SMILES syntax umol accepts. As required by homoiconicity, any leading positions or tagged predicates may contain the full value lattice for the respective attribute type, including concrete values (`#h3`), sets (`#h{1,2}`), or wildcards (`#h*`). For finite sets, set exclusions are also valid syntax (`!H`).

Reactions are written in the same notation. A reaction rule is a left-hand side molecule together with an ordered list of `:deltas` that describe the reaction. Figure 4 illustrates the notation for the proton transfer reaction $NH_3 + HCl \longrightarrow NH_4^+ + Cl^-$, which consists of N – H bond formation, H – Cl bond break, and changes to atomic charges. The right-hand side follows by applying the deltas in order. We discuss the rule composition and application in Section 10.

```
{:lhs {:atoms [[:N "N#h3"] [:H "H"] [:Cl "Cl"]]
       :bonds [{:id :hcl :atoms [:H :Cl] :attrs :single}]}
 :deltas [{:bond {:add [:N :H :single]}} {:bond {:remove :hcl}}
          {:atom {:modify [:N "#c+"]}} {:atom {:modify [:Cl "#c-"]}}]}
```

Figure 4. The proton transfer $NH_3 + HCl \longrightarrow NH_4^+ + Cl^-$ as a reaction rule: a left-hand side molecule (ammonia together with hydrogen chloride) and an ordered list of deltas that add the N – H bond, remove the H – Cl bond, and set the two formal charges. The right-hand side is obtained by applying the deltas in order.

4 UMOL Primer

UMOL is implemented as a Rust library with Python bindings. The Python interface tracks the Rust implementation closely, the only deviations allowed are for writing more idiomatic Python. The key data types are `Molecule` and `Reaction`. A molecule pattern is also a `Molecule`, and a reaction rule is also a `Reaction`. This section gives a short tour of a few common tasks: reading a SMILES string and writing the notation of Section 3, searching for a substructure, computing a fingerprint, and combining, splitting, applying, and composing reactions. Structures are read from SMILES, from the MOL and SDF formats of the chemical table file family [3], and from the notation of Section 3. The examples are given in Python; the corresponding Rust calls are collected in Appendix A.

Getting started. The Python package is published on PyPI and can be installed using `pip`, after which it is imported as `umol`.

```
pip install umol-py
```

The Rust crates and their dependency declarations are given in Appendix A. The version at the time of writing is 0.10.0; UMOL is in alpha testing, and the interfaces described here may still change.

Reading SMILES. A molecule is read from SMILES representation with the `from_smiles` class method, which parses the input and then resolves it under a default chemistry model (Section 7). Resolution fills in the implicit details, for example, atom charge, number of unpaired electrons and implicit hydrogen atoms. Resolution takes advantage of the attribute lattice and is described in more detail in Section 8. Passing `resolve=False` skips resolution.

```
from umol import Molecule

mol = Molecule.from_smiles("CCO")   # ethanol: 3 atoms, 2 bonds
unresolved = Molecule.from_smiles("CCO", resolve=False)
```

Reading UMOL notation. Structures whose bonding does not fit the localized bond model have no SMILES representation. Diborane contains two multicenter (3c-2e) bonds and can be read from the notation of Section 3 that specifies the multicenter bonds overlays, each consisting of the list of atom participants and their respective electron counts.

```
diborane = Molecule.parse('''
  {:atoms ["B" "H" "B" "H" "H" "H" "H" "H"]
   :bonds [[0 4 "1"] [0 5 "1"] [2 6 "1"] [2 7 "1"]]
   :multicenter-bonds [{:atoms [0 1 2] :attrs "[1,1,0]"}
                       {:atoms [0 3 2] :attrs "[0,1,1]"}]}''')
```

The four terminal B – H bonds are localized bonds. Section 5 expands on the representation and manipulation of multicenter bonds, dative/coordination bonds, and noncovalent bonds.

Writing UMOL notation. Rendering a molecule back to the notation of Section 3 is the inverse of the parsing operation.

```
text = mol.render()                    # str(mol) is equivalent
```

The keywords and atom aliases included in the notation (Figure 3) are omitted by the unqualified rendering command. To preserve them across a round trip, parsing and rendering carry them as explicit metadata.

```
mol, metadata = Molecule.parse_with_metadata(text)
text = mol.render_with_metadata(metadata)
```

Writing SMILES. The `to_smiles` method renders the molecule as SMILES representation, while projecting the overlays on atoms and bonds according to the default chemistry model (Section 7). Features not supported by SMILES produce an error.

```
smiles = mol.to_smiles()
```

Depiction as SVG. UMOL offers a simple black-and-white SVG molecule depiction that supports atom annotations, tetrahedral wedges and hashes, cis/trans geometry, and aromatic-system contours. The molecule layout is performed by the BSD-licensed CoordGen library by Schrödinger [49].

```
from pathlib import Path

adenine = Molecule.from_smiles("NC1=NC=NC2=C1N=CN2")  # 9H-adenine, Kekule form
```

```
drawing = adenine.depict()
Path("adenine.svg").write_text(drawing.render_svg(), encoding="utf-8")
```

Choosing a chemistry model. Reading from SMILES representation may also supply a chemistry model to be used in resolution. The chemistry model defines the universe of valid molecules, including accepted atomic valence states, aromaticity models, and allowed types of stereo elements, see Section 7. The example below excludes pentavalent phosphorus, in contrast to the permissive default chemistry model. The atom-typing candidate source enumerates the admissible valence states of each element, so an atom outside the enumeration is a contradiction; the counts-based source instead treats its table entries as saturation targets that govern implicit hydrogen filling, and reads over-target input as saturated.

```
from umol import (
    AtomTypeRegistry, ChemistryModel, ValenceCandidateSource, ValenceModel,
    ValenceTieBreak,
)

STRICT_REGISTRY = """
[P]
0 = ["P #n #v3"]

[F]
0 = ["F #n3 #v"]
"""

default = ChemistryModel.default()
strict = ChemistryModel(
    connectivity=default.connectivity,
    valence=ValenceModel(
        candidates=ValenceCandidateSource.AtomTyping(
            registry=AtomTypeRegistry.from_toml(STRICT_REGISTRY)
        ),
        tie_break=ValenceTieBreak.Strict,
    ),
    aromaticity=default.aromaticity,
    stereo=default.stereo,
)

Molecule.from_smiles("FP(F)F", chemistry_model=strict)          # accepted
Molecule.from_smiles("FP(F)(F)(F)F", chemistry_model=strict)    # umol.ContradictionError:
                          # no atom-typing match for AtomId(1) (element P, charge Some(0))
```

Substructure search. A pattern is an ordinary molecule with open attribute definitions (Section 3); matching it against a host returns the occurrences, each represented by a correspondence map between the pattern entities and the host entities they matched, including overlays. The example below distinguishes the two nitrogen atoms of the imidazole ring in L-histidine by their aromatic valence, which is two for the pyrrole and one for the pyridine nitrogen atom.

```
histidine = Molecule.from_smiles("O=C([C@H](Cc1c[nH]cn1)[NH3+])[O-]")

pyrrole_n  = Molecule.parse('{:atoms ["N#a2"] :bonds []}')  # contributes two pi electrons
pyridine_n = Molecule.parse('{:atoms ["N#a"] :bonds []}')   # contributes one
aromatic_n = Molecule.parse('{:atoms ["N#a+"] :bonds []}')  # any aromatic contribution
any_n      = Molecule.parse('{:atoms ["N"] :bonds []}')

len(pyrrole_n.substructure_matches(histidine))      # 1
len(pyridine_n.substructure_matches(histidine))     # 1
len(aromatic_n.substructure_matches(histidine))     # 2
len(any_n.substructure_matches(histidine))          # 3, including ammonium
```

Each match is a correspondence; `match.atoms.matched_pairs` lists the pattern-to-host atom pairs, and the per-family maps `match.bonds`, `match.aromatic_systems`, and so on relate the remaining entities. Correspondences reverse and compose, so a chain of operations can be followed from end to end. The overlay families themselves are developed in Section 5, the constraints in Section 6, and the definition of matching within the attribute lattice is described in Section 8. Instead of treating the choice of algorithm as an implementation detail, UMOL makes it explicit and allows algorithm selectors for the matching algorithm and the subgraph isomorphism algorithm to be passed as configuration. Depending on the specific molecular task, different algorithmic choices might be preferable. The configuration chosen below uses the incidence (Levi) graph [30] of the molecular structure for the substructure match, and the Ray–Kirsch algorithm [50] for the subgraph isomorphism enumeration.[3]

```
from umol import (SubstructureSearchConfig, SubstructureMatchAlgorithm,
                  SubgraphIsomorphismAlgorithm)

config = SubstructureSearchConfig(
    match_algorithm=SubstructureMatchAlgorithm.Incidence(),
    subgraph_isomorphism_algorithm=SubgraphIsomorphismAlgorithm.RayKirsch(),
)
matches = aromatic_n.substructure_matches(histidine, config=config)
```

[3] The other subgraph isomorphism selectors are VF2 [51] and its optimized variant based on the RDKit implementation[5], RI [52], ArcMatch [53], and the Ullmann algorithm [54]. The use of the Ray–Kirsch algorithm for molecular substructure searches was proposed by Mayfield and Sayle [55].

Fingerprints. Here, we construct the extended-connectivity fingerprint [56] for two isomeric molecules, and compute their Tanimoto similarity. The similarity coefficients used for comparing fingerprints are reviewed in [57].

```
from umol import HashedFingerprintConfig

ecfp = HashedFingerprintConfig.Ecfp(radius=2)    # ECFP4
a = Molecule.from_smiles("CCO").hashed_fingerprint(config=ecfp).fold(2048)
b = Molecule.from_smiles("COC").hashed_fingerprint(config=ecfp).fold(2048)
similarity = a.tanimoto(b)                       # 0.1111
```

The published ECFP algorithm leaves both the fingerprint width and the hashing scheme open. In UMOL the two are separate operations: atom environments are hashed into feature identifiers, and folding to a bit vector of a chosen width follows. The identifiers are 64-bit XXH3 digests [58] taken with a fixed seed.[4] The scheme is machine-independent and versioned: a change to it appears as a new name rather than as a silent difference in identifiers.

Combining and splitting. A reaction in UMOL transforms a left-hand (reactant) side into a right-hand (product) side. Either side may consist of several disconnected components. Reactions of higher molecularity are handled by combining the reactants into a left-hand side graph prior to the reaction application and recovering the products by splitting the right-hand side graph into disconnected components, as described in Section 10.

```
ammonia = Molecule.from_smiles("N")
hydrogen_chloride = Molecule.from_smiles("Cl")
combined = ammonia.combine(hydrogen_chloride)    # one structure, two components
components = combined.split()                    # two molecules
```

Reaction application. A reaction (Figure 4) applies to the concrete (host) molecule, yielding one product molecule per match. The reaction may be constructed from a reaction SMILES, as shown below for the Paal–Knorr synthesis, or from the UMOL notation. The mutating operations (deltas) of the reaction and their application are described in Section 10.

```
from umol import Reaction

# one carbonyl oxygen becomes the ring oxygen; the other leaves as water
rule = Reaction.from_reaction_smiles(
    "[CH3:1][C:2](=[O:3])[CH2:4][CH2:5][C:6](=[O:7])[CH3:8]"
```

[4] The scheme is named `Xxh3Width64V1`. Every digest is taken with the fixed seed `0xECF05EED00000001`. The seed identifier of an atom is the digest of its Daylight invariants written little-endian; each refinement round then digests the round index, the current identifier, and the neighboring (bond, identifier) pairs, also little-endian.

```
    ">>[CH3:1][c:2]1[cH:4][cH:5][c:6]([CH3:8])[o:3]1.[OH2:7]"
)

diketone = Molecule.from_smiles("CC(=O)CCC(=O)C")    # hexane-2,5-dione
products = list(rule.apply(diketone))
product, correspondence = list(rule.tracked_apply(diketone))[0]
                                                     # product and correspondence
```

The reaction SMILES is resolved into deltas, including the addition of the furan aromatic system `{:aromatic-system {:add {:atoms [1 3 4 5 2] :attrs "[1,1,1,1,2]#c0#u0#s"}}}`. Tracked application returns a correspondence between the host and product.

Applying a reaction produces the right-hand side as a single graph, all reaction applications are unimolecular and have a single product. The `react` operation applies the reaction and splits the result, returning the product components of each match. For several reactants, `react_all` combines them first and runs the same pipeline in one call.

```
product_sets = list(diketone.react(rule))    # one list of products per match
len(product_sets[0])                          # 2 (furan and water)
```

Composing reactions. Two reactions compose into a list of composite reactions according to the rule algebra shown in Section 10, one for each admissible overlap between the right-hand (product) side of the first reaction and the left-hand (reactant) side of the second.

```
composites = first.compose(second)    # one reaction per admissible overlap
```

Applying the composites to a molecule gives the same products as applying the first reaction and then the second. Collecting both sets of products and comparing them:

```
sequential = {str(product.canonicalize()) for intermediate in first.apply(host)
                                          for product in second.apply(intermediate)}
composed = {str(product.canonicalize()) for composite in composites
                                        for product in composite.apply(host)}
sequential == composed    # True
```

The two collections agree as sets of products. They need not agree in multiplicity: a symmetric substrate or a repeated overlap can reach the same product by more than one route on one side and not on the other.

Table 2. The entity kinds of a molecular structure with their participants and attributes.

Kind	Participants	Attributes
Atom	—	element, isotope, charge, implicit H, lone pairs, spin
Bond	two atoms	order, charge, spin
Dative bond	donors, one acceptor	order
Aromatic system	set of atoms	per-atom π electrons, charge, spin
Multicenter bond	set of atoms	per-atom electrons, charge, spin
Noncovalent bond	two atoms	kind
Stereo atom	site atom, ordered ligands	class, configuration
Stereo bond	site bond, ordered ligands	class, configuration

5 Molecular Topology and Overlays

The distinction between the topology and overlays is fundamental, as Figure 1 shows. The molecular topology is defined by atoms and localized covalent bonds, while all other structural features are lifted into overlays. With this definition, molecular topology is an attributed undirected simple graph, and the algebraic laws of graphs, including graph rewriting, apply to it without qualification. Each overlay kind can be understood as an attributed hyperedge, or, alternatively, as a relation set (table). Each table row is an entity consisting of foreign keys into the molecular topology features (atoms and bonds), which are referred to as *participants*, and a set of attributes (columns) [42, 43]. The entity kinds differ in the organization of the participant set; the participants of aromatic systems and multicenter bonds are equivalent and unordered; the ligands in stereo atoms and stereo bonds form an ordered sequence; the participants of dative bonds and stereo atoms and bonds split into distinct roles (donors and acceptor for dative bonds, stereo site and ligands for stereo atoms and bonds). As mentioned in the introduction, folding localized bonds into the relational structure would increase the uniformity of the model, but at the cost of efficient graph traversal, substructure search, and rewriting algorithms, and was rejected.

Table 2 lists the entity kinds, that is, atoms, bonds, and six overlay kinds. Aromatic systems and multicenter bonds share a shape but differ in the chemistry they describe and in their perception algorithms. Noncovalent bonds are characterized as overlays despite their binary shape because their dissociation energies and chemical properties are distinct from those of localized covalent bonds. The ordering of the stereo ligands provides a fixed frame for defining the configuration index. A ligand frame may name a position that is not an atom of the graph: an implicit hydrogen, written `[:h a]`, or a lone pair, `[:lp a]`. Lone-pair ligands participate in the stereochemistry of sulfoxides [24].

Figure 3 shows the topology and two overlays (aromatic system and stereo atom) of the L-histidine zwitterion. The stereo ligands include an implicit hydrogen atom. The diborane representation in Section 4 features two multicenter bonds. Figure 5 includes two aromatic systems and two dative (coordination) bonds. The aromatic system specifies the atomic π contributions and its delocalized charge. We note that the carbon atoms of the cyclopentadienyl ligands are equivalent, in contrast to the SMILES representation `[cH-]1cccc1`, which must localize the charge on an arbitrary carbon atom,

```
{:atoms [[:Fe "Fe#c+2#n3"]
         [:C1 "C#h"] [:C2 "C#h"] [:C3 "C#h"] [:C4 "C#h"] [:C5 "C#h"]
         [:C6 "C#h"] [:C7 "C#h"] [:C8 "C#h"] [:C9 "C#h"] [:C10 "C#h"]]
 :bonds [[:C1 :C2 :single] [:C2 :C3 :single] [:C3 :C4 :single]
         [:C4 :C5 :single] [:C5 :C1 :single]
         [:C6 :C7 :single] [:C7 :C8 :single] [:C8 :C9 :single]
         [:C9 :C10 :single] [:C10 :C6 :single]]
 :aromatic-systems [{:atoms [:C1 :C2 :C3 :C4 :C5]  :attrs "[1,1,1,1,1]#c-"}
                    {:atoms [:C6 :C7 :C8 :C9 :C10] :attrs "[1,1,1,1,1]#c-"}]
 :dative-bonds [{:donors [:C1 :C2 :C3 :C4 :C5]  :acceptor :Fe :attrs "3"}
                {:donors [:C6 :C7 :C8 :C9 :C10] :acceptor :Fe :attrs "3"}]}
```

Figure 5. Ferrocene $Fe(C_5H_5)_2$ in the UMOL notation. The structure contains two aromatic systems and two dative bonds. #n3 records the six paired *d* electrons of low-spin iron(II). The cyclopentadienyl ligands feature symmetric atomic π contributions (`[1,1,1,1,1]`) and a delocalized negative charge (#c-). Each ligand donates 3 electron pairs (6 electrons).

breaking the ring symmetry. The total number of electrons in the aromatic system is the sum of the atomic contributions minus the delocalized charge. A potential refinement of the representation model would allow relations between relations, in this case, between an aromatic system and an atom. The practicality of this additional indirection (reification) is open at present.

6 Identity and Constraints

An entity carries attributes of two kinds: *inherent fields* and *constraints*. Inherent fields such as the element of an atom, its charge, the order of a bond, or the per-atom π contributions of an aromatic system were discussed in the previous section, see Table 2 for the full listing. Only inherent fields define an entity's identity, and they are always present, although their values may be undetermined. In contrast, a *constraint* is an assertable fact about an entity, which leaves its identity unchanged. A constraint need not be present at all, and an undetermined constraint is equivalent to its absence. Most constraints are derivable from the relational structure, as Table 3 shows. The diagonal entries contain the inherent fields, while the off-diagonal entries display the constraints that one entity kind derives from its incidence with another. Only the atom and bond rows are populated, since relations between overlays are not part of the model.

A derivable constraint has a derived side and an asserted side. On a concrete molecule its value is obtained by projection, so that the aromatic valence of an atom is read from the aromatic system that contains it; in a pattern the same constraint is asserted without needing to reproduce the exact relational structure. The derived side is the analog of the SMILES string, and the asserted side is the corresponding SMARTS property. The derived and asserted constraints behave identically with respect to attribute matching; in both cases, the attribute lattice defines the matching rules, see Section 8. Asserted constraints can be used to denote patterns or express relational structure in external formats that do not

Table 3. Inherent fields (diagonal) and derivable constraints (off-diagonal) of entity kinds, shown in the UMOL notation of Section 3. `#v` is the valence of an atom (sum of incident bond orders), `#D` is atom degree (number of bonds), `#d` and `#t` are donated and accepted pair counts, `#m` is multicenter valence. `#a` denotes aromatic valence (π contribution) for atoms and incidence to aromatic systems for bonds. `#T` and `#C` are the projections of the tetrahedral stereo atom and cis–trans stereo bond onto the stereo site, respectively. Noncovalent bonds provide no derivable constraints at present and are omitted.

	atom	bond	dative	aromatic	multicenter	stereo
atom	`#i#c#h#n#u#s`	`#v#D`	`#d#t`	`#a`	`#m`	`#T`
bond		order `#c#u#s`	—	`#a`	—	`#C`
dative			order	—	—	—
aromatic				electrons `#c#u#s`	—	—
multicenter					electrons `#c#u#s`	—
stereo						class, configuration

```
{:atoms [[:n "N#a1#h0#R(6)1"] [:c1 "C#a1"] [:c2 "C#a1"]]
 :bonds [[:c1 :n "1#a"] [:n :c2 "1#a"]]}
```

Figure 6. A pattern for a pyridine-type nitrogen. The nitrogen carries an inherent field (`#h0`, no implicit hydrogen), a constraint derived from an aromatic system (`#a1`, one π electron contributed), and one derived from the ring set (`#R(6)`, one ring of six).

support overlays.

In addition to projections of the relational structure onto its participants, UMOL defines a set of constraints familiar from the SMARTS vocabulary [4] such as degree `#D`, total degree `#X`, total valence `#V`, total hydrogens `#H`, ring degree `#x`, and ring membership `#R`. Figure 6 is a pattern for a pyridine-type nitrogen whose atom string carries an inherent field and two constraints: `#h0` is inherent, `#a1` is derivable from an aromatic system, and `#R(6)1` from the ring set. It matches pyridine and the pyridine-type nitrogen of quinoline, and rejects pyrrole on the hydrogen count and imidazole on the ring size, whose nitrogen is otherwise of the same type.

The ring constraints require the definition of the ring set. UMOL fixes the reference ring set as the set of relevant rings, which corresponds to the union of all minimum cycle bases [59]. The maximum ring size included in the reference ring set is fixed at 22. The smallest set of smallest rings (SSSR) [60] is not used because it is not unique [61].

Two structures are the same when one is a relabeling of the other (Principle 3). Identity and hashing of molecular structure thus require canonical labeling. UMOL uses nauty, from McKay and Piperno's nauty and Traces project [45], for graph automorphism computation and canonical labeling. As discussed above, only inherent fields contribute to the entity labels (colors) used in the canonical labeling. Since canonical labeling is a computationally expensive procedure, canonicalization is lazy, that is, it is only applied on request. Two readings of ethanol that differ in the order of their atoms are distinct under structural (ordinary) equality but equal under canonical equality.

```
ethanol = Molecule.from_smiles("CCO")
same    = Molecule.from_smiles("OCC")

ethanol == same              # False -- the atoms are stored in a different order
ethanol.canonical_eq(same)   # True -- one is a relabeling of the other
```

Reactions are canonicalized by the same operation. The identifiers a reaction's deltas use belong to the reaction itself (Section 10), so canonicalization selects a frame for those identifiers and transports the left-hand side and the deltas into it. Two rules that differ only in their identifiers then share a canonical form, and reactions are compared and hashed like molecules.

7 Validity and Chemistry Model

A valid molecule is one which the user might wish to include in their work; all other molecules are considered invalid, even if they are representable by the notation or the in-memory data structures, see Principle 7. In this sense, validity is fundamentally a *model-dependent* property. Validity is often equated with molecule stability, which is tied to physical conditions and experimental timescales. In evaluating validity, UMOL distinguishes between physical invariants and conformance to a specific model of valence, aromaticity, and stereochemistry, among others. The invariants must be fulfilled unconditionally. Physical invariants include electron conservation per atom, which relates the atom charge, its number of unpaired electrons and lone pairs to the atomic contributions to localized bonds, dative bonds, aromatic systems, and multicenter bonds. This is the extension of the electron counting schemes of Lewis and Langmuir [62]. On the other hand, a valence model might exclude open-shell or hypervalent compounds and an aromaticity model might restrict the element scope [63] and choose Hückel rule [64–66] or the Clar model [67, 68] of aromaticity. The model specifications are bundled in the *chemistry model*, which can be specified on input or used for explicit validation. An example is given in Section 4. We will consider additional examples in the following.

The model is data. Figure 7 is an excerpt from the atom-typing valence model, which enumerates the admissible states of each element and charge as atom strings in the notation of Section 3. Conformance to the valence model can utilize the attribute lattice: an atom must be compatible with one of the rows, that is, there exists a concrete element of the lattice that is included in both the input atom specification and one of the admissible atom types. We discuss the attribute lattice in Section 8. In addition to the atom typing model, a counts-based model is provided, which follows the approach of the built-in validation scheme in RDKit [5]. The counts model utilizes a valence table, which provides target covalences for the admissible elements, see example in Section 4.

The chemistry model is consulted during resolution, which attempts to impute the atomic valence states (including implicit hydrogen atoms, unpaired electrons, and lone pairs), aromatic systems, and stereo atoms and bonds from the partial specification given by external formats such as SMILES. If the provided input is insufficient to fix the molecular structure, resolution fails with an `Underdetermined` result, while an inconsistent input produces a `Contradictory` failure mode. Resolution may also be skipped on ingestion; the unstated attributes then remain undetermined. Resolution does not apply

```
[N]
0 = [
    "N #n #v3",       # N2, NH3, NX3, HONO, N2O3, HN=NH, H2N–NH2
    "N #n #v2 #a",    # C5H5N
    "N #v3 #a2",      # C4H4NX, C4H4NH
    "N #u #v2 #a2",   # pyrrolyl radical
    "N #n2 #v",       # NX, NH
    "N #n #u3",       # N atom, s2p3
]
1 = [
    "N #c+ #v4",      # [NX4]+, [NH4]+, [NO2]+, HONO2
]
-3 = [
    "N #c-3 #n4",     # N3-
]
```

Figure 7. Selection of admissible nitrogen states from the atom-typing valence model. The atom types use the notation of Section 3.

kekulization, nor does it convert implicit hydrogen annotations into explicit atoms; those operations alter the molecular constitution and must be invoked separately by the user. Projection creates the constraints again from the resolved structure.

8 Attribute Lattice

An attribute lattice describes the available, potentially partial information about an attribute, which may range from concrete values, also referred to as *ground*, to the undetermined value (wildcard). We encounter partial information in two distinct contexts. An incomplete description, for example from external input, is filled by resolution; a pattern (query) matches multiple concrete values. Resolution is addressed in Section 7; we discuss matching in this section. For two members of the lattice a and b, we write $a \prec b$ and draw an ascending line from a to b if all values a admits are also admitted by b, see also Principle 5 and Figure 2. The wildcard $*$ admits all values ($a \prec *$ for any a) and is thus the top of the lattice. The empty set is included in every set and is the bottom member of the lattice, denoted as $\bot$.

The lattice operations formalize resolution and substructure matching. Whether a pattern a admits a value b is matching (Principle 5). Whether two partial descriptions a and b can be satisfied at once is compatibility. Both are defined in terms of two operations, the *meet* $a \wedge b$ and the *join* $a \vee b$. Meet and join are the greatest lower bound and least upper bound; for element sets, these are intersection and union. a matches b when $a \wedge b = b$. Two members of the lattice a and b are compatible if $a \wedge b \neq \bot$, that is, some concrete value is admitted by both of them. Table 4 gives the lattice operations on the element lattice and the lattice of aromatic valence values. Aromatic valence (`#a`) is an integer value, but `#a0` and `#a!` express different constraints. An atom with `#a0` belongs to an aromatic system and contributes no electrons to the delocalized set, as boron does in borepin C_6H_7B; an atom with `#a!` belongs to no aromatic system,

Table 4. Lattice operations: meet $a \wedge b$, join $a \vee b$, match, and compatibility. a and b are members of the element lattice (upper panel) and aromatic valence lattice (#a, lower panel). $*$ is the element wildcard. $\perp$ marks an empty meet when a and b are incompatible. Atoms belonging to an aromatic system have #an with $n \geq 0$, #a! asserts that the atom belongs to no aromatic system, and #a* leaves aromatic valence undetermined.

a	b	$a \wedge b$	$a \vee b$	a matches b	compatible
{C,N}	{N,O}	N	{C,N,O}	no	yes
{C,N,O}	N	N	{C,N,O}	yes	yes
*	C	C	*	yes	yes
C	N	$\perp$	{C,N}	no	no
#a*	#a1	#a1	#a*	yes	yes
#a{1,2}	#a2	#a2	#a{1,2}	yes	yes
#a1	#a2	$\perp$	#a{1,2}	no	no
#a!	#a1	$\perp$	#a*	no	no

as boron does in boric acid $B(OH)_3$. The results in the top row of Table 4 can be computed as follows.

```
from umol import Element, ElementForm

C_N = ElementForm.LitSet({Element("C"), Element("N")})
N_O = ElementForm.LitSet({Element("N"), Element("O")})

C_N.meet(N_O)             # N -- the only element both admit
C_N.join(N_O)             # {C,N,O} -- the least description admitting either
C_N.matches(N_O)          # False -- {N,O} admits O, and {C,N} does not
C_N.is_compatible(N_O)    # True -- N satisfies both
```

In Section 7, we described resolution as refinement, which moves the input downward within the lattice. Two failure modes of refinement are contradiction, when no ground value satisfies both the input and the model, and undetermined result, when the model fails to sufficiently refine the input to reach a ground value.

9 Mutation

Reactions are a particular instance of manipulating molecules. Construction, transformations, and reactions rely on a shared molecule mutation infrastructure, using *edits* as the unit of change. Edits are again data and can be inspected, stored, and replayed. Building methylamine from ammonia takes three edits: adding a carbon atom, attaching it to the nitrogen by a single bond, and updating the nitrogen's implicit hydrogen count.

```
from umol import AtomForm, AtomUpdate, BondForm, Edits, Molecule

ammonia = Molecule.from_smiles("N")
```

```
edits = Edits()
carbon = edits.add_atom(AtomForm.parse("C#h3"))
edits.add_bond(0, carbon, BondForm.parse("1"))
edits.update_atom(0, AtomForm.parse("N#h3"), AtomUpdate.parse("#h2"))

methylamine = ammonia.apply(edits)
```

No chemistry rules are enforced when applying edits: without the third edit, the implicit hydrogen count remains unchanged. Validating the reaction products under a chemistry model is a separate operation. The same sequence of edits written in the notation of Section 3 is

```
[{:atom {:add "C#h3"}}
 {:bond {:add [0 {:new 0} :single]}}
 {:atom {:modify [0 {:expect "#h3" :update "#h2"}]}}]
```

An entity created earlier in the same sequence is referred to as `{:new 0}`, since it has no position in the original structure; adding an entity returns that handle. The returned handle can be used to refer to the added entity in the subsequent edits. Applying a batch of edits produces a new molecule with the requested changes, leaving the original molecule unchanged. If application fails, an exception is raised.

Edits provide an imperative mutation interface for a given host molecule. They name the entities already present in the host and generate new handles if necessary. A reaction must describe a change to any molecule matched by its left-hand side. The reaction representation operates on a stable, combined set of entity identifiers from the left-hand side and the deltas, as we discuss in Section 10.

10 Reactions

In UMOL a reaction is encoded by a left-hand side (LHS) molecule and an ordered list of deltas, which describe the applied changes. Only the LHS is stored; the right-hand side follows by applying the deltas in order. The LHS may be a concrete molecule or a pattern, so reactions and reaction rules share one representation (Principle 4). Figure 4 is such a reaction, and Section 4 shows an example of applying a reaction to a host molecule. Deltas add and remove entities as well as modify them, so the number of atoms and other entities need not be conserved. The reaction below attaches a methyl group to ammonia.

```
{:lhs {:atoms [[:N "N#h3"]] :bonds []}
 :deltas [{:atom {:add [:C "C#h3"]}}
          {:bond {:add [:N :C :single]}}
          {:atom {:modify [:N "#h2"]}}]}
```

The identifier `:C` is introduced by a delta and does not occur in the LHS, so the correspondence between the two sides of the reaction is partial. Only the atoms carried over from the LHS are related to atoms of the product.

The same three changes are written as edits in Section 9. An edit names an entity by its position in the host it will be applied to, or by a handle such as `{:new 0}` for an entity added earlier in the same sequence. A delta names an entity by an identifier that belongs to the reaction. The LHS supplies those identifiers, and a delta that adds an entity extends the same set, as `:C` does above. A reaction is therefore self-contained. Deltas invert and normalize, and reactions reverse and compose without a concrete molecule available. Edits offer none of these operations, because their positional references are resolved only when a host is supplied.

A reaction may equivalently be written as the superimposed graph of its two sides, in which each entity is either unchanged or carries the verb that changes it. The condensed graph of reaction [69, 70], reaction SMARTS and SMIRKS [4], and the left, context, and right sections of MØD rule files [10] all use that form. The two forms are interconvertible, and a reaction can be built from its two sides together with an atom correspondence, so an existing collection of rules need not be rewritten by hand.

A reaction is applied by matching its LHS against the host. Each match converts the deltas into edits on the matched entities, which are then applied as one batch to a copy of the host. Each applicable match yields a product molecule; *tracked* application also returns a correspondence from the host to the product. The extended dangling condition of Principle 9 constrains the rule: the deletion of an atom while leaving one of its bonds or overlays is rejected.

Reactions compose. Composition of rules over the common subgraphs of their facing sides is the concurrent rule construction of algebraic graph transformation [32], developed for chemical reaction rules in MØD [10, 71]. Given two reactions applied in sequence, UMOL returns one composite reaction for every way the second can reuse what the first produced, and the two are not required to share anything. Composing the ionization of methyl bromide with the capture of the resulting cation by water gives two composites. In the first, the second reaction consumes the cation that the first produced, and the composite is the substitution, with three atoms on its LHS. In the second the overlap is empty: the composite ionizes one molecule while capturing a cation that was already present, and its LHS is the two patterns side by side, four atoms in total. Two steps that touch unrelated parts of a molecule are therefore not a special case but the overlap of size zero. An overlap is admissible when the attributes of the shared entities are compatible (Section 8) and neither application leaves an entity dangling.

11 Conclusion

The commitments of Section 2 carry trade-offs. The separation of topology and overlays is required for the algebraic properties of graphs and relations to hold, and the attribute lattice enables a consistent treatment of all entities with respect to matching and reactions. However, the result is eight entity kinds instead of two (atoms and bonds) and simple integers wrapped in a lattice type to express an atomic charge. The notation, while more expressive, is considerably longer than the SMILES string for a molecule of similar size. A compact interface or an economical notation are not the primary goal of the UMOL library design, which prioritizes a strict separation of concerns and an explicit specification. These trade-offs are motivated by the assumption that convenient interfaces can be built on explicit and verifiable behaviors, and not the reverse. The UMOL library interface thus demands more of the user: resolution requires a chemistry model, substructure search algorithms carry an algorithm selector, and

canonicalization must be explicitly requested. The utility of this assumption is, like all chemical theory, subject to experimental validation.

Acknowledgments

I owe a debt of gratitude to present and former Dayhoff Labs colleagues, specifically, Steve Crossan, Josh Goldford, John Hymel, Toni Lee, Sam Maddrell-Mander, Sam Marks, Daniel Martin-Alarcon, Alex Mathiasen, Derek Metcalf, Jason Rocks, Harrison Smith, Olga Taran, Dat Truong, and Dariia Yehorova, for their support and interest in this project. Financial support by Dayhoff Labs is gratefully acknowledged.

Tool Use

The implementation of a large library would not have been possible without the participation of coding agents, specifically GPT-5.5 and GPT-5.6 Sol and Claude Sonnet and Opus 3.5 through 5. I reviewed all code commits; any program bugs and errors are mine.

Availability

UMOL is available at `https://github.com/dayhofflabs/umol` under a permissive license (dual Apache-2.0 and MIT). This paper describes version 0.10.0. The Python bindings used in the listings are distributed as `umol-py` and imported as `umol`; the normative specification of the notation summarized in Appendix C is distributed with the library.

References

1. Weininger, D. SMILES, a chemical language and information system. 1. Introduction to methodology and encoding rules. *J. Chem. Inf. Comp. Sci.* **28,** 31–36. doi:10.1021/ci00057a005 (1988).
2. Blue Obelisk. *OpenSMILES* `http://opensmiles.org`. Accessed 2026-07-17.
3. Dalby, A. *et al.* Description of several chemical structure file formats used by computer programs developed at Molecular Design Limited. *J. Chem. Inf. Comp. Sci.* **32,** 244–255. doi:10.1021/ci00007a012 (1992).
4. Daylight Chemical Information Systems. *Daylight Theory Manual* `https://www.daylight.com/dayhtml/doc/theory/`. Accessed 2026-07-16.
5. Landrum, G. & RDKit contributors. *RDKit: Open-source cheminformatics* `https://www.rdkit.org`. Accessed 2026-07-16.
6. ChemAxon. *Marvin and JChem* `https://www.chemaxon.com`. Accessed 2026-07-16.
7. O'Boyle, N. M. *et al.* Open Babel: An open chemical toolbox. *J. Cheminf.* **3,** 33. doi:10.1186/1758-2946-3-33 (2011).
8. Willighagen, E. L. *et al.* The Chemistry Development Kit (CDK) v2.0: atom typing, depiction, molecular formulas, and substructure searching. *J. Cheminf.* **9,** 33. doi:10.1186/s13321-017-0220-4 (2017).

9. EPAM Systems. *Indigo Toolkit* https://lifescience.opensource.epam.com/indigo/. Accessed 2026-07-17.
10. Andersen, J. L., Flamm, C., Merkle, D. & Stadler, P. F. *A Software Package For Chemically Inspired Graph Transformation* in *Graph Transformation* (eds Echahed, R. & Minas, M.) (Springer, Cham, 2016), 73–88. doi:10.1007/978-3-319-40530-8_5.
11. American Chemical Society. *CAS SciFinder* https://scifinder-n.cas.org/. Accessed 2026-07-17.
12. National Library of Medicine. National Center for Biotechnology Information. *PubChem* https://pubchem.ncbi.nlm.nih.gov. Accessed 2026-07-17.
13. Elsevier Ltd. *Reaxys* https://www.reaxys.com/. Accessed 2026-07-17.
14. Royal Society of Chemistry. *ChemSpider* https://www.chemspider.com/. Accessed 2026-07-17.
15. Gaulton, A. *et al.* ChEMBL: a large-scale bioactivity database for drug discovery. *Nucleic Acids Res.* **40,** D1100–D1107. doi:10.1093/nar/gkr777 (2012).
16. Irwin, J. J., Tang, K. G., Young, J., *et al.* ZINC20—A Free Ultralarge-Scale Chemical Database for Ligand Discovery. *J. Chem. Inf. Model.* **60,** 6065–6073. doi:10.1021/acs.jcim.0c00675 (2020).
17. Blum, L. C. & Reymond, J.-L. 970 Million Druglike Small Molecules for Virtual Screening in the Chemical Universe Database GDB-13. *J. Am. Chem. Soc.* **131,** 8732–8733. doi:10.1021/ja902302h (2009).
18. Ruddigkeit, L., van Deursen, R., Blum, L. C. & Reymond, J.-L. Enumeration of 166 billion organic small molecules in the chemical universe database GDB-17. *J. Chem. Inf. Model.* **52,** 2864–2875. doi:10.1021/ci300415d (2012).
19. Lowe, D. M. *Chemical reactions from US patents (1976–Sep2016)* 2017. doi:10.6084/m9.figshare.5104873.v1.
20. Schreiner, M., Bhowmik, A., Vegge, T., Busk, J. & Winther, O. Transition1x – a dataset for building generalizable reactive machine learning potentials. *Sci. Data* **9,** 779. doi:10.1038/s41597-022-01870-w (2022).
21. Levine, D. S., Shuaibi, M., Spotte-Smith, E. W. C., Taylor, M. G., Hasyim, M. R., *et al. The Open Molecules 2025 (OMol25) Dataset, Evaluations, and Models* arXiv:2505.08762. https://arxiv.org/abs/2505.08762. 2025.
22. Sylvester, J. J. Chemistry and Algebra. *Nature* **17,** 284. doi:10.1038/017284a0 (1878).
23. Pólya, G. Kombinatorische Anzahlbestimmungen für Gruppen, Graphen und chemische Verbindungen. *Acta Math.* **68,** 145–254. doi:10.1007/BF02546665 (1937).
24. IUPAC. Rules for the nomenclature of organic chemistry. Section E: Stereochemistry. *Pure Appl. Chem.* **45,** 11–30. doi:10.1351/pac197645010011 (1976).
25. Heller, S. R. & McNaught, A. D. The IUPAC International Chemical Identifier (InChI). *Chem. Int.* **31,** 7–9. doi:10.1515/ci.2009.31.1.7 (2009).

26. Rupp, M., Tkatchenko, A., Müller, K.-R. & von Lilienfeld, O. A. Fast and Accurate Modeling of Molecular Atomization Energies with Machine Learning. *Phys. Rev. Lett.* **108,** 058301. doi:10.1103/PhysRevLett.108.058301 (2012).

27. Ramakrishnan, R., Dral, P. O., Rupp, M. & von Lilienfeld, O. A. Quantum chemistry structures and properties of 134 kilo molecules. *Sci. Data* **1,** 140022. doi:10.1038/sdata.2014.22 (2014).

28. Grambow, C. A., Pattanaik, L. & Green, W. H. Reactants, products, and transition states of elementary chemical reactions based on quantum chemistry. *Sci. Data* **7,** 137. doi:10.1038/s41597-020-0460-4 (2020).

29. Wood, B. M. *et al. UMA: A Family of Universal Models for Atoms* arXiv:2506.23971. https://arxiv.org/abs/2506.23971. 2025.

30. Berge, C. *Graphs and Hypergraphs* (North-Holland, Amsterdam, 1973).

31. Davey, B. A. & Priestley, H. A. *Introduction to Lattices and Order* 2nd ed. doi:10.1017/CBO9780511809088 (Cambridge University Press, Cambridge, 2002).

32. Ehrig, H., Ehrig, K., Prange, U. & Taentzer, G. *Fundamentals of Algebraic Graph Transformation* doi:10.1007/3-540-31188-2 (Springer, Berlin Heidelberg, 2006).

33. Ugi, I. *et al.* in *Computer Chemistry* 199–233 (Springer, Berlin, Heidelberg, 1993). doi:10.1007/BFb0111463.

34. Dugundji, J. & Ugi, I. in *Computers in Chemistry* 19–64 (Springer, Berlin, Heidelberg, 1973). doi:10.1007/BFb0051317.

35. Dietz, A. Yet Another Representation of Molecular Structure. *J. Chem. Inf. Comp. Sci.* **35,** 787–802. doi:10.1021/ci00027a001 (1995).

36. Goldstein, O. & March, S. *Representing Molecules with Algebraic Data Types: Beyond SMILES and SELFIES* arXiv:2501.13633. https://arxiv.org/abs/2501.13633. 2025.

37. Orejas, F. Symbolic graphs for attributed graph constraints. *J. Symb. Comput.* **46,** 294–315. doi:10.1016/j.jsc.2010.09.009 (2011).

38. Höck, S. & Riedl, R. chemf: A purely functional chemistry toolkit. *J. Cheminf.* **4,** 38. doi:10.1186/1758-2946-4-38 (2012).

39. Cayley, A. LVII. On the mathematical theory of isomers. *Philos. Mag.* **47,** 444–447. doi:10.1080/14786447408641058 (1874).

40. Balaban, A. T. Applications of graph theory in chemistry. *J. Chem. Inf. Comput. Sci.* **25,** 334–343. doi:10.1021/ci00047a033 (1985).

41. *Chemical Graph Theory. Introduction and Fundamentals* (eds Bonchev, D. & Rouvray, D. H.) (Gordon and Breach, New York, 1991).

42. Codd, E. F. A relational model of data for large shared data banks. *Commun. ACM* **13,** 377–387. doi:10.1145/362384.362685 (1970).

43. Abiteboul, S., Hull, R. & Vianu, V. *Foundations of Databases* (Addison-Wesley, Reading, MA, 1995).
44. Diestel, R. *Graph Theory* 5th ed. doi:10.1007/978-3-662-53622-3 (Springer, Berlin, Heidelberg, 2017).
45. McKay, B. D. & Piperno, A. Practical graph isomorphism, II. *J. Symb. Comput.* **60,** 94–112. doi:10.1016/j.jsc.2013.09.003 (2014).
46. Hickey, R. A history of Clojure. *Proc. ACM Program Lang.* **4,** 1–46. doi:10.1145/3386321 (2020).
47. Hickey, R. *edn: Extensible Data Notation* https://github.com/edn-format/edn. Accessed 2026-07-16.
48. Cognitect, I. *Datomic Pro. The fully transactional, cloud-ready, distributed database* https://datomic.com. Accessed 2026-07-18.
49. Schrödinger Inc. *Schrodinger-developed 2D Coordinate Generation* https://github.com/schrodinger/coordgenlibs. Accessed 2026-09-05.
50. Ray, L. C. & Kirsch, R. A. Finding Chemical Records by Digital Computers. *Science* **126,** 814–819. doi:10.1126/science.126.3278.814 (1957).
51. Cordella, L. P., Foggia, P., Sansone, C. & Vento, M. A (sub)graph isomorphism algorithm for matching large graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* **26,** 1367–1372. doi:10.1109/TPAMI.2004.75 (2004).
52. Bonnici, V., Giugno, R., Pulvirenti, A., Shasha, D. & Ferro, A. A subgraph isomorphism algorithm and its application to biochemical data. *BMC Bioinformatics* **14,** S13. doi:10.1186/1471-2105-14-S7-S13 (2013).
53. Bonnici, V., Grasso, R., Micale, G., *et al.* ArcMatch: high-performance subgraph matching for labeled graphs by exploiting edge domains. *Data Min. Knowl. Discov.* **38,** 4076–4116. doi:10.1007/s10618-024-01061-8 (2024).
54. Ullmann, J. R. An Algorithm for Subgraph Isomorphism. *J. ACM* **23,** 31–42. doi:10.1145/321921.321925 (1976).
55. Mayfield, J. & Sayle, R. *The Secrets of Fast SMARTS Matching* 8th Joint Sheffield Conference on Chemoinformatics, https://www.nextmovesoftware.com/talks/Mayfield_SecretsOfFastSmartsMatching_Sheffield_201906.pdf. Accessed 2026-07-28. 2019.
56. Rogers, D. & Hahn, M. Extended-Connectivity Fingerprints. *J. Chem. Inf. Model.* **50,** 742–754. doi:10.1021/ci100050t (2010).
57. Willett, P., Barnard, J. M. & Downs, G. M. Chemical Similarity Searching. *J. Chem. Inf. Comp. Sci.* **38,** 983–996. doi:10.1021/ci9800211 (Nov. 1998).
58. Collet, Y. *xxHash fast digest algorithm* https://github.com/Cyan4973/xxHash/blob/dev/doc/xxhash_spec.md. Specification; accessed 2026-07-27.
59. Vismara, P. Union of all the minimum cycle bases of a graph. *Electron. J. Comb.* **4,** R9. doi:10.37236/1294 (1997).

60. Plotkin, M. Mathematical basis of ring-finding algorithms in CIDS. *J. Chem. Doc.* **11,** 60–63. doi:10.1021/c160040a013 (1971).

61. Berger, F., Flamm, C., Gleiss, P. M., Leydold, J. & Stadler, P. F. Counterexamples in chemical ring perception. *J. Chem. Inf. Comput. Sci.* **44,** 323–331. doi:10.1021/ci030405d (2004).

62. Langmuir, I. The arrangement of electrons in atoms and molecules. *J. Am. Chem. Soc.* **41,** 868–934. doi:10.1021/ja02227a002 (1919).

63. Sciences, O. C. M. *OEChem Toolkit. Theory. Aromaticity Perception* https://docs.eyesopen.com/toolkits/python/oechemtk/aromaticity.html. Accessed 2026-08-01.

64. Hückel, E. Quantentheoretische Beiträge zum Benzolproblem: II. Quantentheorie der induzierten Polaritäten. *Z. Phys.* **72,** 310–337. doi:10.1007/bf01341953 (1931).

65. Hückel, E. Quantentheoretische Beiträge zum Problem der aromatischen und ungesättigten Verbindungen. III. *Z. Phys.* **76,** 628–648. doi:10.1007/bf01341936 (1932).

66. Hückel, E. Grundzüge der Theorie ungesättigter und aromatischer Verbindungen. *Z. Elektrochem. Angew. Phys. Chem.* **43,** 752–788. doi:10.1002/bbpc.19370430907 (1937).

67. Clar, E. *The Aromatic Sextet* ISBN: 0-471-15840-2 (Wiley, London, 1972).

68. Liu, M. & Green, W. H. Capturing aromaticity in automatic mechanism generation software. *Proc. Combust. Inst.* **37,** 575–581. doi:10.1016/j.proci.2018.06.006 (2019).

69. Fujita, S. Description of organic reactions based on imaginary transition structures. 1. Introduction of new concepts. *J. Chem. Inf. Comput. Sci.* **26,** 205–212. doi:10.1021/ci00052a009 (1986).

70. Nugmanov, R. I. *et al.* CGRtools: Python Library for Molecule, Reaction, and Condensed Graph of Reaction Processing. *J. Chem. Inf. Model.* **59,** 2516–2521. doi:10.1021/acs.jcim.9b00102 (2019).

71. Andersen, J. L., Flamm, C., Merkle, D. & Stadler, P. F. Inferring Chemical Reaction Patterns Using Rule Composition in Graph Grammars. *J. Syst. Chem.* **4,** 4. doi:10.1186/1759-2208-4-4 (2013).

72. Bron, C. & Kerbosch, J. Algorithm 457: finding all cliques of an undirected graph. *Commun. ACM* **16,** 575–577. doi:10.1145/362342.362367 (1973).

73. McGregor, J. J. Backtrack search algorithms and the maximal common subgraph problem. *Softw. Pract. Exp.* **12,** 23–34. doi:10.1002/spe.4380120103 (1982).

Appendix A The Rust Interface

The Python bindings track the Rust library closely, and almost every operation of Section 4 has a Rust counterpart. This appendix collects the corresponding calls and illustrates their use. For brevity, the Rust code examples are written as snippets from `Result`-returning functions and thus use the compact try operator `?`. If used in free-standing scripts or outside of error-handling functions, the latter should be replaced by `unwrap()` method calls or explicit error handling.

UMOL is organized as a workspace of crates, of which five cover the operations shown here: `umol-graph`, which implements ingestion, matching, and fingerprints; `umol-graph-ir`, which provides the underlying data types for molecules, reactions, and their attributes; `umol-graph-core`, which provides the graph algorithms; `umol-edn`, which implements the EDN notation reader and writer; and `umol-io`, which provides format readers for external formats and depictions.

```
[dependencies]
umol-edn = "0.10.0"
umol-graph = "0.10.0"
umol-graph-core = "0.10.0"
umol-graph-ir = "0.10.0"
umol-io = "0.10.0"
```

Reading SMILES. `ingest_smiles` and `ingest_smiles_unresolved` read a SMILES string with and without resolution.

```
use umol_graph::ingest;

let mol = ingest::ingest_smiles("CCO")?;
let unresolved = ingest::ingest_smiles_unresolved("CCO")?;
```

Reading UMOL notation. Structures in the notation are read with the standard `parse` method.

```
use umol_graph_ir::ir::Molecule;

let diborane: Molecule = r#"
  {:atoms ["B" "H" "B" "H" "H" "H" "H" "H"]
   :bonds [[0 4 "1"] [0 5 "1"] [2 6 "1"] [2 7 "1"]]
   :multicenter-bonds [{:atoms [0 1 2] :attrs "[1,1,0]"}
                       {:atoms [0 3 2] :attrs "[0,1,1]"}]}"#.parse()?;
```

Writing UMOL notation. `to_string` writes a molecule in the notation. Converting it to `MoleculeDsl` with `MoleculeDefaults::concrete()` gives a more compact form that leaves out default values.

```
use umol_graph_ir::dsl::{MoleculeDefaults, MoleculeDsl};
use umol_graph_ir::ir::FromIr;

let text = mol.to_string();
let compact = MoleculeDsl::from_ir(&mol, &MoleculeDefaults::concrete()).to_string();
```

`MoleculeDsl` also holds the keywords and atom aliases, so they survive a round trip.

```
use umol_edn::FromEdn;

let (mol, metadata) = MoleculeDsl::from_edn_str(&text)?.into_parts();
let text = MoleculeDsl::new(mol, metadata)?.to_string();
```

Writing SMILES. SMILES is written with `export::export_smiles`.

```
use umol_graph::export;

let smiles = export::export_smiles(&mol)?;
```

Choosing a chemistry model. `ingest_smiles_with` reads SMILES under a given chemistry model and also takes the SMILES input and resolution settings. The listing builds the strict model of Section 4.

```
use std::borrow::Cow;
use umol_graph::ingest::ingest_smiles_with;
use umol_graph::ops::model::{
    ChemistryModel, ValenceCandidateSource, ValenceModel, ValenceTieBreak,
};
use umol_graph::ops::resolve::ResolveConfig;
use umol_graph::ops::valence::AtomTypeRegistry;
use umol_io::smiles::SmilesIoConfig;

let registry = AtomTypeRegistry::from_toml_str(
    "[P]\n0 = [\"P #n #v3\"]\n\n[F]\n0 = [\"F #n3 #v\"]",
)?;
let strict = ChemistryModel {
    valence: ValenceModel {
        candidates: ValenceCandidateSource::AtomTyping { registry: Cow::Owned(registry) },
        tie_break: ValenceTieBreak::Strict,
    },
    ..ChemistryModel::default()
};
```

```rust
let io = SmilesIoConfig::opensmiles();
let accepted = ingest_smiles_with("FP(F)F", &io, &strict, &ResolveConfig::default())?;
let refused = ingest_smiles_with("FP(F)(F)(F)F", &io, &strict, &ResolveConfig::default());
// Err(Contradiction: no atom-typing match)
```

Substructure search. In Rust, substructure search and reaction application require an explicit algorithm configuration: the matching representation, the subgraph isomorphism algorithm, and the ring perception algorithm (Section 6).

```rust
use umol_graph_core::{RelevantCycleEnumerationAlgorithm, SubgraphIsomorphismAlgorithm};
use umol_graph_ir::ir::{SubstructureMatchAlgorithm, SubstructureMatchConfig};

let pattern: Molecule = r#"{:atoms ["O"] :bonds []}"#.parse()?;
let config = SubstructureMatchConfig {
    match_algorithm: SubstructureMatchAlgorithm::GraphAndOverlays,
    subgraph_isomorphism_algorithm: SubgraphIsomorphismAlgorithm::Vf2,
    relevant_cycle_algorithm: RelevantCycleEnumerationAlgorithm::Vismara,
};
let matches = pattern.substructure_matches(&mol, config)?;
```

Fingerprints. Fingerprints are computed with `EcfpFeaturizer`, then folded and compared as in Python.

```rust
use umol_graph::fingerprint::EcfpFeaturizer;

let ecfp = EcfpFeaturizer::new(2);    // ECFP4
let a = ecfp.featurize(&ingest::ingest_smiles("CCO")?)?.fold(2048)?;
let b = ecfp.featurize(&ingest::ingest_smiles("COC")?)?.fold(2048)?;
let similarity = a.tanimoto(&b)?;
```

Combining and splitting. The calls are the same as in Python.

```rust
let ammonia = ingest::ingest_smiles("N")?;
let hydrogen_chloride = ingest::ingest_smiles("Cl")?;
let combined = ammonia.combine(&hydrogen_chloride);
let components = combined.split();    // one product molecule per component
```

Reaction application. Reaction application takes the same configuration as substructure search.

```rust
use umol_graph::ingest;

let rule = ingest::ingest_reaction_smiles(concat!(
    "[CH3:1][C:2](=[O:3])[CH2:4][CH2:5][C:6](=[O:7])[CH3:8]>>",
    "[CH3:1][c:2]1[cH:4][cH:5][c:6]([CH3:8])[o:3]1.[OH2:7]"))?;
let diketone = ingest::ingest_smiles("CC(=O)CCC(=O)C")?;
for product in rule.apply(&diketone, config)? {
    println!("{}", product?);
}
for application in rule.tracked_apply(&diketone, config)? {
    let (product, correspondence) = application?;
    println!("{}", product);
}
```

`react` accepts a single molecule or an array of reactants, so Rust needs no separate `react_all`. Each match yields a list of products.

```rust
use umol_graph_ir::ir::React;

for products in diketone.react(&rule, config)? {
    let products = products?;    // one Vec<Molecule> per match
}
let with_reagent = [first, second].react(&rule, config)?;
```

Composing reactions. In Rust, `compose` takes the common subgraph algorithm as an argument; Python uses a default. The overlaps are found either as cliques of the modular product [72] or by backtracking over partial mappings [73].

```rust
use umol_graph_core::algorithms::common_subgraph::CommonSubgraphEnumerationAlgorithm;

let composites = first.compose(
    &second,
    CommonSubgraphEnumerationAlgorithm::ModularProductBacktracking,
);
```

Appendix B Glossary

Terms are given in the sense used in this paper; the cross-reference points to where each is introduced.

aromatic system
: An overlay representing a delocalized π system. Its attributes include the π-electron contributions of the participating atoms. Principle 2

aromatic valence
: The π-electron contribution of an atom to its aromatic system, for example, one for a pyridine-type and two for a pyrrole-type nitrogen. Principle 6

attribute lattice
: The values an attribute may take (literals, sets of admissible values, and the wildcard), ordered by set inclusion. Principle 5

bond
: An edge of the molecular topology: a localized single, double, or triple bond. Dative, multicenter, and noncovalent bonds are overlays. Principle 1

canonical labeling
: An atom numbering that gives isomorphic structures the same form; used for identity tests and hashing. Principle 3

chemistry model
: The rules that decide validity, such as valence rules, the aromaticity model, and admissible stereo kinds; used in resolution and validation. Principle 7

combine
: Merging two structures into one, for example, the two reactants of a bimolecular reaction. See also split. Section 4

constraint
: A property of an entity, such as degree or ring membership, that does not contribute to its identity. An undetermined constraint is equivalent to an absent one. See also inherent field. Section 6

correspondence
: A map between the entities of two structures, for example, from a pattern to its match in a host, or from a host to a reaction product. Section 4

covalence
: The number of electrons an atom gains by sharing: its localized valence and implicit hydrogen count, plus one if it contributes a single π electron to an aromatic system. The term follows Langmuir [62]. Section 7

dangling condition
: The requirement that a reaction rule deleting an atom also delete every bond and overlay in which that atom participates; extended here from edges to overlays. Principle 9

delta
: A change in a reaction: adding, removing, or modifying an atom, bond, or overlay. Deltas refer to entities by identifiers that belong to the reaction, and every delta has an inverse. Principle 8

double-pushout rewriting
: The construction from algebraic graph transformation that defines how a reaction rule applies to a molecule and how rules compose. Principle 9

edit
: A change to a given molecule that refers to entities by their position in it. Removing an atom also removes its bonds and overlays. A list of edits is applied as a whole or not at all. Section 9

EDN
: The extensible data notation of the Clojure language, used as the outer envelope of the UMOL notation. Section 3

entity
: An atom, bond, or overlay. Section 5

ground term
: A structure in which every attribute has a concrete value, that is, a molecule in the ordinary sense. Principle 5

homoiconicity
: Molecules and patterns share one notation and one data type, and so do concrete reactions and reaction rules. Principle 4

hyperedge
: An edge that joins any number of atoms; the underlying form of an overlay. Principle 2

inherent field
: An attribute that defines an entity's identity, such as element or charge. It is always present, although its value may be undetermined. See also constraint. Section 6

meet, join
: The greatest lower bound and least upper bound of two attribute values: their intersection and union under set inclusion. Principle 5

molecular structure
: A molecular topology together with its overlays. Principle 2

molecular topology
: The attributed undirected simple graph of atoms and localized bonds; used interchangeably with *molecular graph*. Principle 1

overlay
: A typed, attributed hyperedge for a structural feature outside the topology: aromatic systems, stereo

atoms and bonds, and dative, multicenter, and noncovalent bonds. Overlays of one kind form a relation set. Principle 2

participants
: The atoms or bonds that an overlay joins, for example, the ring atoms of an aromatic system or the site and ligands of a stereo atom. Section 5

pattern
: A structure with at least one attribute given as a set of admissible values or a wildcard. Molecules that refine a pattern match it. Principle 4

projection
: The inverse of resolution under a chemistry model, which replaces overlays with constraints on their participating atoms and bonds and omits defaults recoverable by resolution. Section 4

reaction
: A left-hand side together with an ordered list of deltas. The left-hand side is a single structure, possibly with several disconnected components. Principle 9

refinement
: Replacing a value by a more specific one, for example, $\{C, N\}$ by N. Matching and resolution are both defined by refinement. Principle 5

relation set
: All overlays of one kind, read as a table with one row per overlay listing its participants and attributes. Principle 2

resolution
: Filling in what an input format such as SMILES leaves unstated: implicit hydrogens, unpaired electrons, lone pairs, aromatic systems, and stereo atoms and bonds. Resolution is refinement in the attribute lattice. Section 8

split
: The decomposition of a structure into its connected components. Section 4

stereo atom, stereo bond
: Overlays that hold the configuration of a stereocenter or stereogenic bond with respect to an ordered list of ligands. Principle 2

tracked
: Said of splitting and reaction application when they also return a correspondence. Section 10

validity
: Whether a structure is acceptable as a molecule. Physical invariants such as electron balance always apply; valence, aromaticity, and stereo rules come from the chemistry model. Principle 7

Appendix C Notation Reference

This appendix summarizes the notation of Section 3 and lists the hash tags for atom and bond attributes; the normative grammar is distributed with the library. In the grammar below, `::=` introduces a definition, `|` separates alternatives, and a trailing `*` means zero or more repetitions. All other symbols appear as written.

Envelope. A molecule is an EDN map with the required keys `:atoms` and `:bonds` and the optional overlay keys `:dative-bonds`, `:aromatic-systems`, `:multicenter-bonds`, `:noncovalent-bonds`, `:stereo-atoms`, and `:stereo-bonds` (Figure 3). A reaction is a map of a left-hand side and a vector of deltas (Figure 4).

```
molecule-map ::= { :atoms atom-list  :bonds bond-list  overlay-section* }
reaction-map ::= { :lhs molecule-map  :deltas [ delta* ] }
delta        ::= { entity-keyword { :add entry
                                  | :remove ref
                                  | :modify [ ref partial-string ] } }
```

An atom entry is an attribute string, optionally paired with a name in a two-element vector. A bond entry is a vector of two atom references (indices or names) and a bond string. The long form is a map, like the overlay entries, with an optional `:id` key.

```
atom-entry ::= "atom-string" | [ name "atom-string" ]
bond-entry ::= [ atom-ref atom-ref bond-spec ]
             | { :atoms [ atom-ref atom-ref ] :attrs bond-spec }
atom-ref   ::= integer | name
```

Overlay entries follow the map spelling with their own participant keys: `:atoms` for aromatic systems and multicenter bonds, `:donors` and `:acceptor` for dative bonds, `:ligands` and `:site` for stereo entities. Each carries an `:attrs` string.

Values. A value in an attribute string is a literal (`3`, `F`), a set in braces (`{1,2}`), or the wildcard `*`; finite sets can also be written as exclusions (`!H`). These are the members of the value lattice of Table 4.

Attributes. Each attribute string has a leading attribute (element for atoms, numerical order for bonds) and a set of optional attributes (fields and constraints), which are introduced by single-letter hash tags. See Table 5 for atom attribute strings and Table 6 for bond attribute strings. For numerical attributes, the literal `1` may be omitted, that is, `#h1` and `#h` mean the same. Similarly, for the charge attributes, `#c+` and `#c-` denote charges of $+1$ and -1. Some special values use symbols. `#i=` represents the natural isotope composition. The symbol `+` on `#a`, `#m`, `#R`, `#T`, or `#C` asserts the property without fixing its value, and `!` asserts its absence: `#a+` matches any aromatic atom, `#a!` only atoms outside aromatic systems, and `#a*` both. Common bond attribute strings can be represented by keywords, for example, `"1"` is equivalent

to `:single`, `"2"` corresponds to `:double`, and `"1#a"` (single localized bond coincident with an aromatic system) can be written as `:aromatic`.

Table 5. Hash tags for atom attributes.

Tag	Attribute
`#i`	isotope mass; `#i=` for the natural isotopic composition
`#c`	formal charge
`#h`	implicit hydrogen count
`#n`	lone pairs
`#u`	unpaired electrons
`#s`	spin multiplicity, $2S + 1$
`#v`	localized valence: sum of all incident bond orders
`#d`	dative pairs donated
`#t`	dative pairs accepted
`#a`	aromatic π contribution
`#m`	multicenter valence
`#T`	tetrahedral stereo configuration
`#D`	degree: number of neighbors
`#X`	total degree: degree plus implicit hydrogens
`#V`	total valence: localized, implicit, aromatic and multicenter
`#x`	ring degree: number of ring bonds
`#y`	ring valence: bond orders of ring bonds
`#H`	total hydrogens: implicit plus explicit neighbors
`#R`	ring membership; `#R(`n`)` counts rings of size n

Table 6. Hash tags for bond attributes.

Tag	Attribute
`#c`	formal charge on the bond
`#u`	unpaired electrons on the bond
`#s`	spin multiplicity on the bond
`#a`	membership of an aromatic system
`#R`	ring membership; derived
`#C`	cis/trans stereo configuration